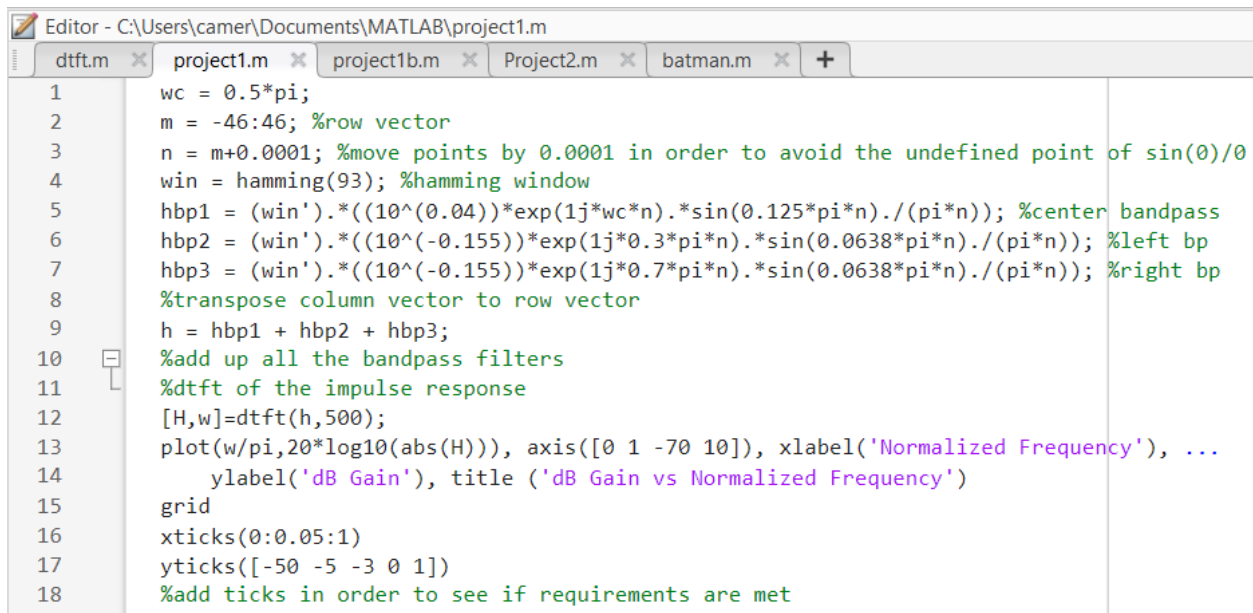


Project 1 – EECS 152B

Cameron Peterson-Zopf

Question 1a: Windowing Method

For this question, I used the hamming window method. I found this window method was superior due to its low sidelobes allowing me to reduce the amount of the samples I needed while maintaining the restriction that the ends of the graph be below -50 dB. My code for this question is below in figure 1.

The image shows a screenshot of the MATLAB Editor window. The title bar indicates the file path is C:\Users\camer\Documents\MATLAB\project1.m. The editor has several tabs open: dtft.m, project1.m (active), project1b.m, Project2.m, and batman.m. The code in project1.m is as follows:

```
1 wc = 0.5*pi;
2 m = -46:46; %row vector
3 n = m+0.0001; %move points by 0.0001 in order to avoid the undefined point of sin(0)/0
4 win = hamming(93); %hamming window
5 hbp1 = (win').*((10^(0.04))*exp(1j*wc*n).*sin(0.125*pi*n)./(pi*n)); %center bandpass
6 hbp2 = (win').*((10^(-0.155))*exp(1j*0.3*pi*n).*sin(0.0638*pi*n)./(pi*n)); %left bp
7 hbp3 = (win').*((10^(-0.155))*exp(1j*0.7*pi*n).*sin(0.0638*pi*n)./(pi*n)); %right bp
8 %transpose column vector to row vector
9 h = hbp1 + hbp2 + hbp3;
10 %add up all the bandpass filters
11 %dtft of the impulse response
12 [H,w]=dtft(h,500);
13 plot(w/pi,20*log10(abs(H)), axis([0 1 -70 10]), xlabel('Normalized Frequency'), ...
14      ylabel('dB Gain'), title ('dB Gain vs Normalized Frequency'))
15 grid
16 xticks(0:0.05:1)
17 yticks([-50 -5 -3 0 1])
18 %add ticks in order to see if requirements are met
```

Figure 1 – Code for Question 1a

Plotting the result, figure 2 displays the overall graph of the magnitude with respect to normalized frequency.

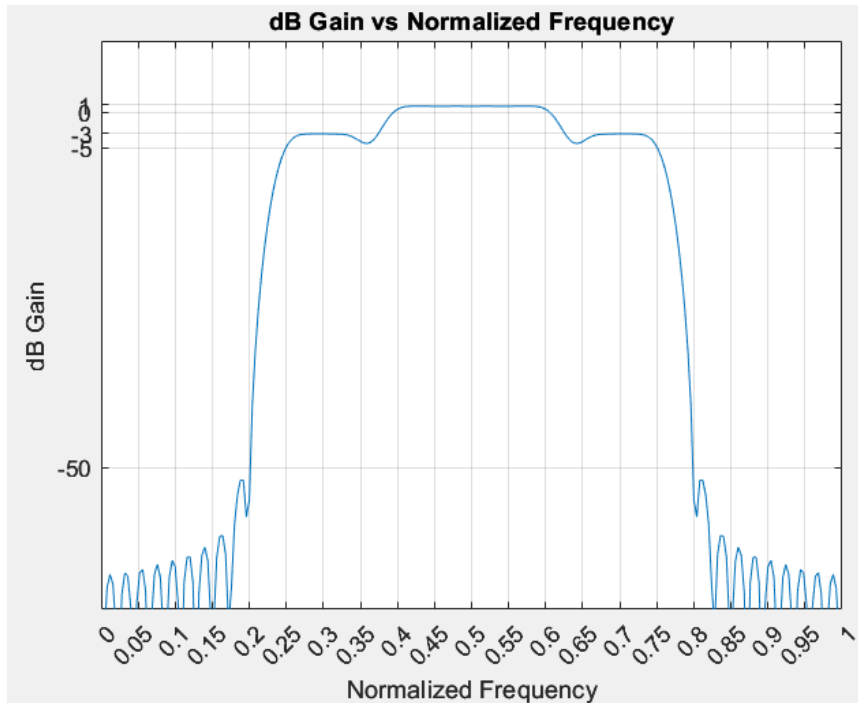


Figure 2 – dB Gain vs Normalized Frequency for Question 1a

Zooming in on each region, the requirements for the ranges of acceptable dB gain are satisfied as can be seen in the following graphs.

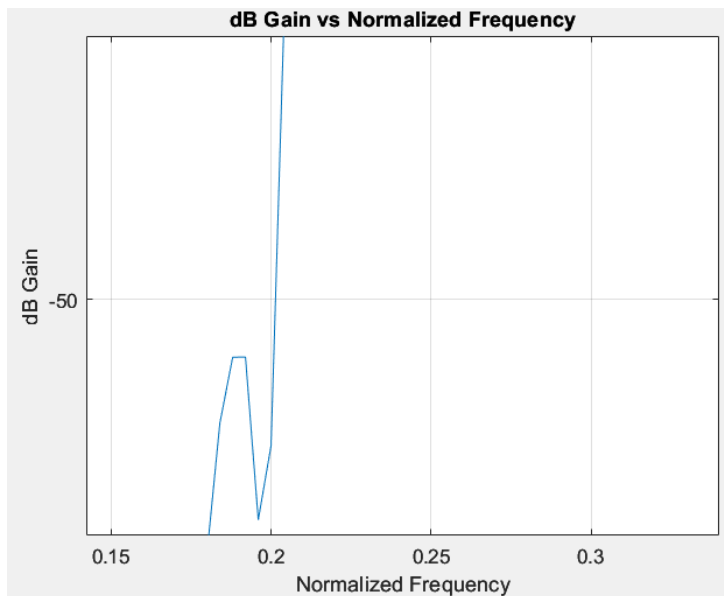


Figure 3 – 0 to 0.2π Criteria

Figure 3 illustrates how the dB gain remains below -50 dB between 0 to 0.2π .

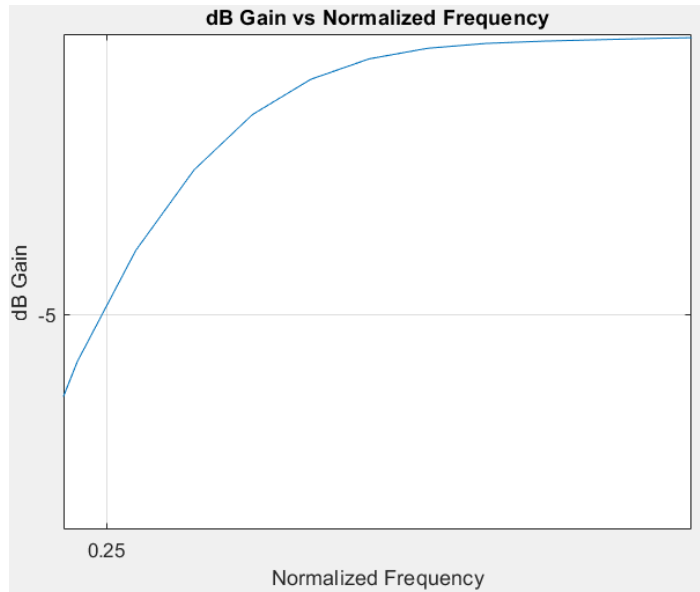


Figure 4 – Criteria at $w = 0.25\pi$

From the graph, it is evident that at $w = 0.25\pi$, the criteria for dB gain between -5 and -3 is met. Furthermore, as seen in figure 5, the entire range between 0.25π and 0.35π remains between -5 and -3 dB.

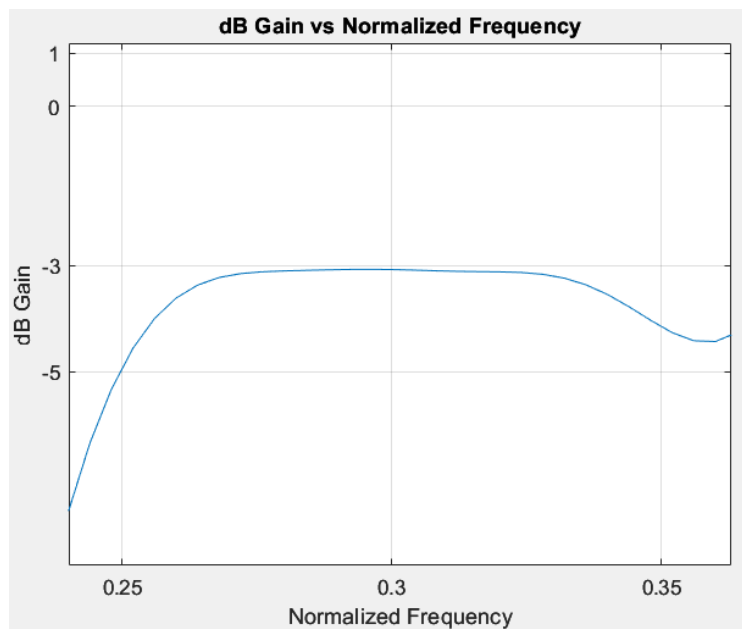


Figure 5 – Criteria between 0.25π and 0.35π

Between 0.4π and 0.6π the graph must be in the range of 0 to 1 dB. Displayed in figure 6 is this region, where these requirements are met.

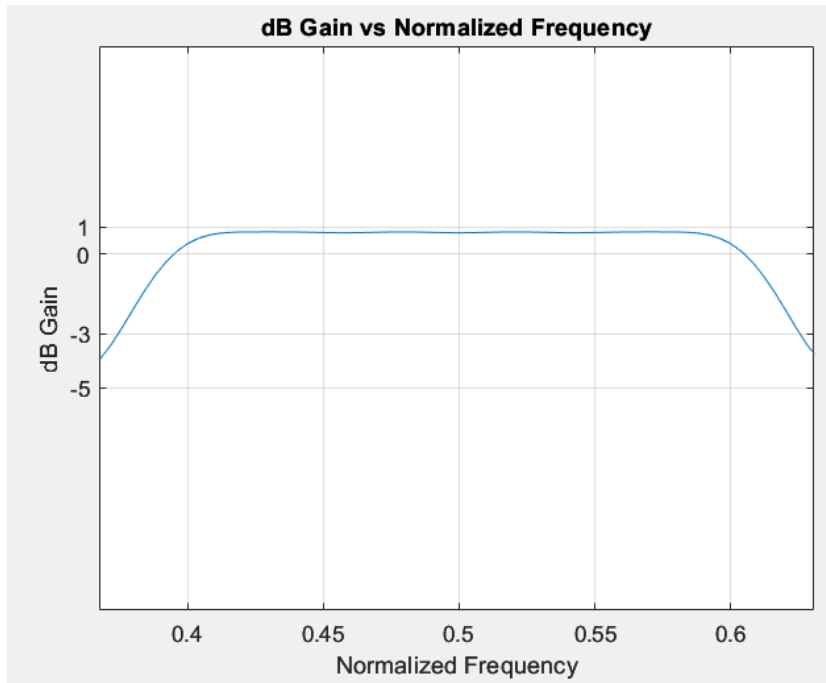


Figure 6 – Criteria for 0.4π to 0.6π

Moving on to 0.65π to 0.75π , this region requires the dB gain between -5 and -3 dB, which is satisfied as can be seen from figure 7.

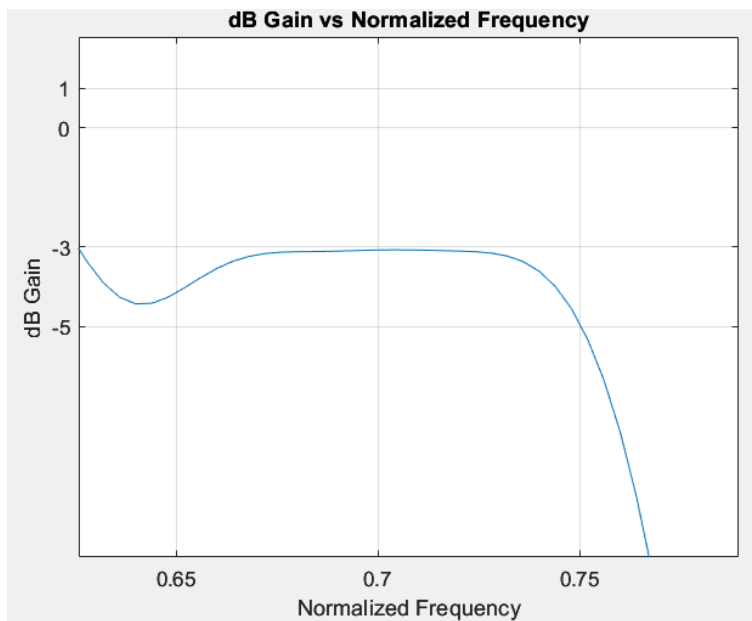


Figure 7 – Criteria between 0.65π and 0.75π

Zooming in on 0.75π , we see that at this point the dB gain is above the -5 dB threshold as seen in figure 8.

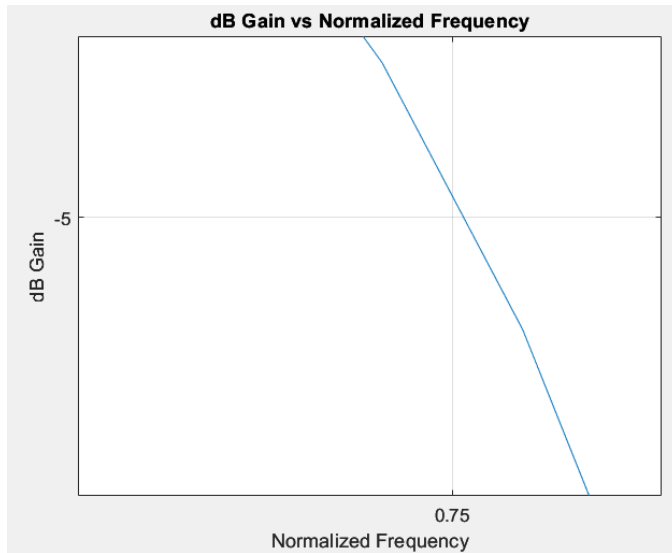


Figure 8 - Criteria at 0.75pi

Lastly, the requirements are satisfied at the 0.8pi – pi range. Figure 9 displays how the 0.8pi frequency is below the -50 dB threshold.

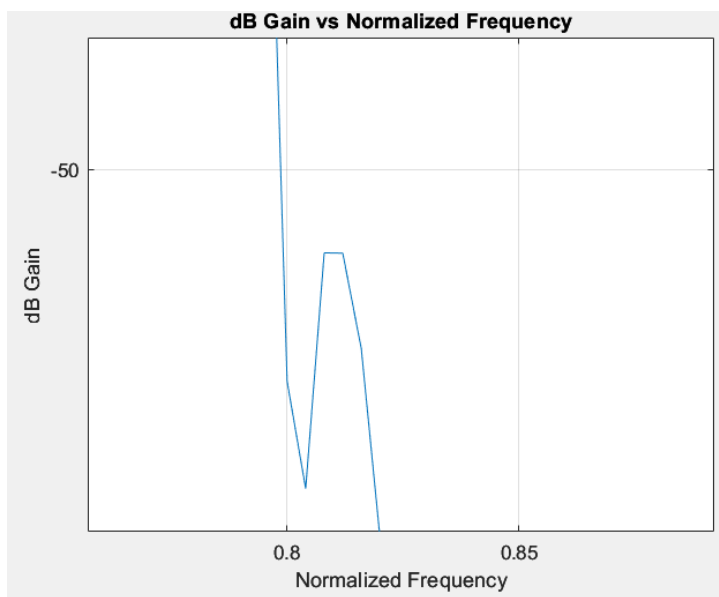


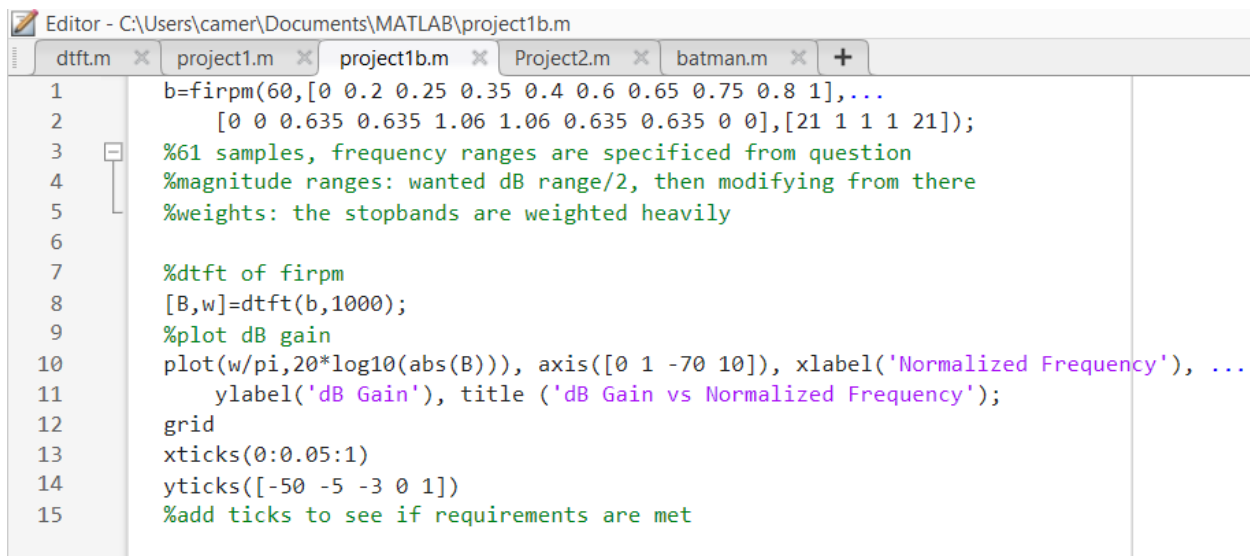
Figure 9 - Criteria at 0.8pi

As displayed, the minimum number of samples required to meet the criteria was 93. Below 93 samples, I could not create a filter where the requirements were satisfied. To create the filter, I added 3 bandpass filters together, where each bandpass was constructed through a shifted low pass filter. The sinc functions create the low pass filters, with the complex exponential in the formulas serving to shift the center point of the low pass filter to create the

bandpass filter. The hamming window was utilized to dampen the sidelobes in order to meet the -50 dB threshold while maintaining a reasonable slope in the transition band. I found through experimenting with the other bands that either the sidelobes were too big or the transition band's slope was too shallow, meaning the -5 dB threshold could not be met by 0.25π .

Question 1b: Parks-McClellan Method

The Parks-McClellan algorithm utilizes an algorithm to find the optimal Chebyshev FIR filter. This algorithm utilizes weights in order to bias reducing the error in certain sections rather than others, adding flexibility for the user in order to target stopbands and passbands. Through trial and error, I found that the least number of samples required to meet the specifications was 61. I determined what gain specifications to give my filter based off of initially inputting in the mean value of the range of satisfactory dB gains, and then modifying these slightly to get down to 61 samples. I weighted the stopband heavily in order to reduce the sidelobes. The resulting code is displayed below in figure 10.



```
Editor - C:\Users\camer\Documents\MATLAB\project1b.m
dtft.m  project1.m  project1b.m  Project2.m  batman.m  +
1      b=firpm(60,[0 0.2 0.25 0.35 0.4 0.6 0.65 0.75 0.8 1],...
2          [0 0 0.635 0.635 1.06 1.06 0.635 0.635 0 0],[21 1 1 1 21]);
3      %61 samples, frequency ranges are specified from question
4      %magnitude ranges: wanted dB range/2, then modifying from there
5      %weights: the stopbands are weighted heavily
6
7      %dtft of firpm
8      [B,w]=dtft(b,1000);
9      %plot dB gain
10     plot(w/pi,20*log10(abs(B))), axis([0 1 -70 10]), xlabel('Normalized Frequency'), ...
11         ylabel('dB Gain'), title ('dB Gain vs Normalized Frequency');
12     grid
13     xticks(0:0.05:1)
14     yticks([-50 -5 -3 0 1])
15     %add ticks to see if requirements are met
```

Figure 10 – Code for Question 1b

Plotting the above code yields the following graph in figure 11.

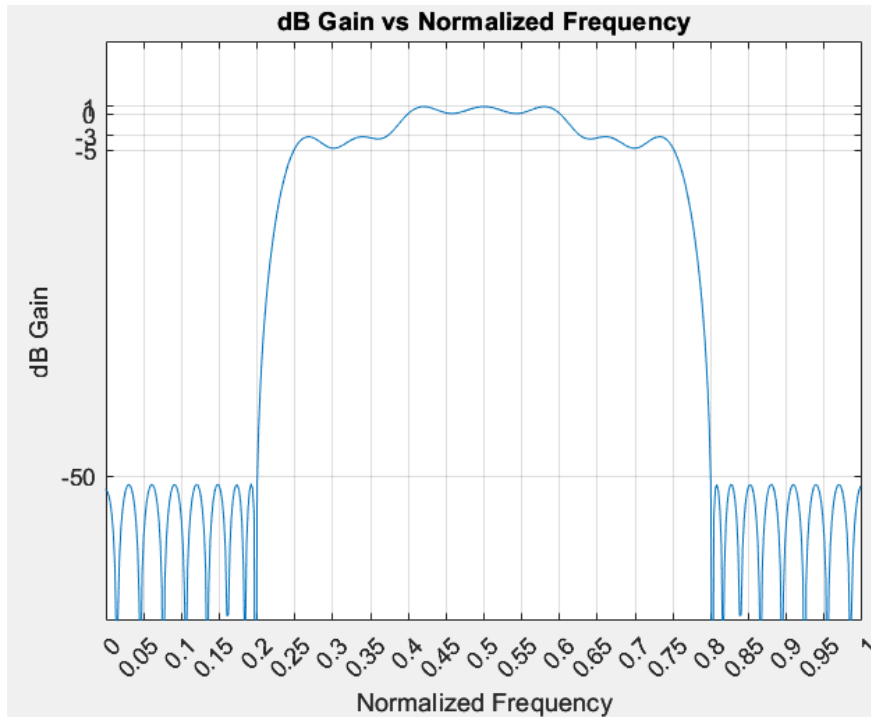


Figure 11 – dB Gain vs Normalized Frequency for Question 1b

Zooming in on each region, the requirements for the ranges of acceptable dB gain are satisfied as can be seen in the following graphs.

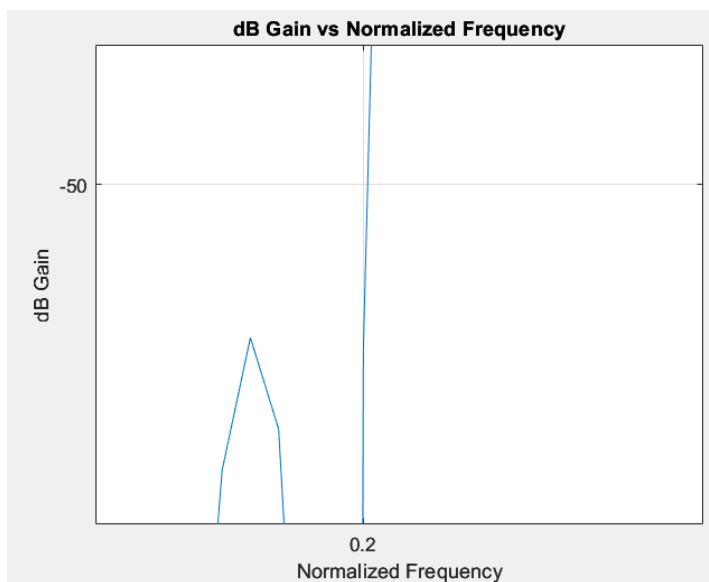


Figure 12 – Criteria for 0.2π

At 0.2π , the requirement of below -50 dB gain is met as seen in figure 12. Furthermore, the -5 dB to -3 dB range is satisfied in the 0.25π to 0.35π region.

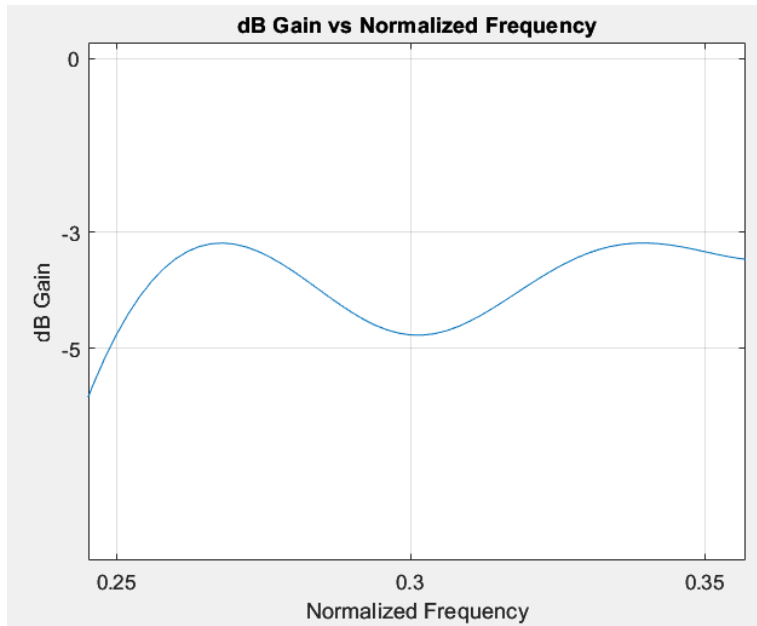


Figure 13 – Criteria for 0.25π to 0.35π

Furthermore, figures 14, 15 and 16 illustrate that the graph maintains a range between 0 and 1 dB between 0.4π and 0.6π .

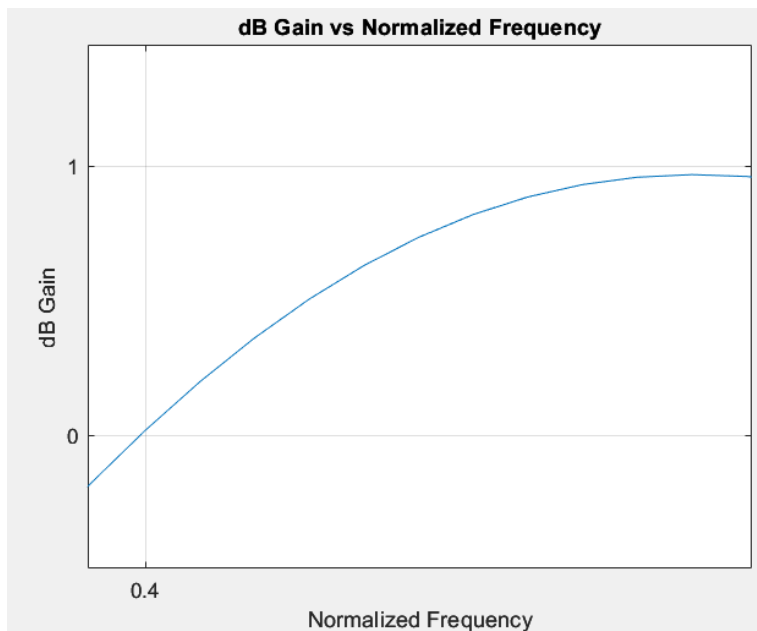


Figure 14 – Criteria for 0.4π

In addition, as we the ripples of the region we see that none of them go outside of the wanted range as demonstrated in figure 15.

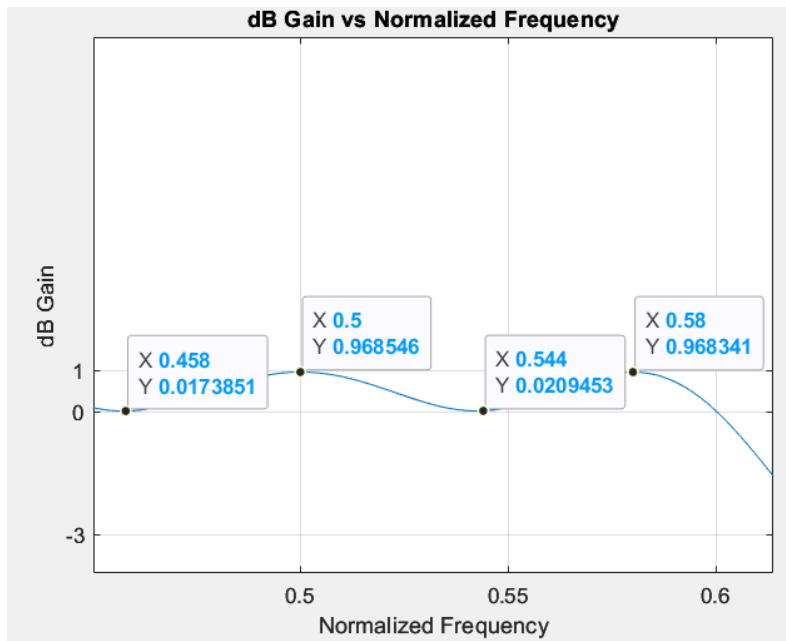


Figure 15 – 0.4pi to 0.6pi Region

Analyzing the 0.6pi point with more zoom, we see that this point is above the 0 dB threshold.

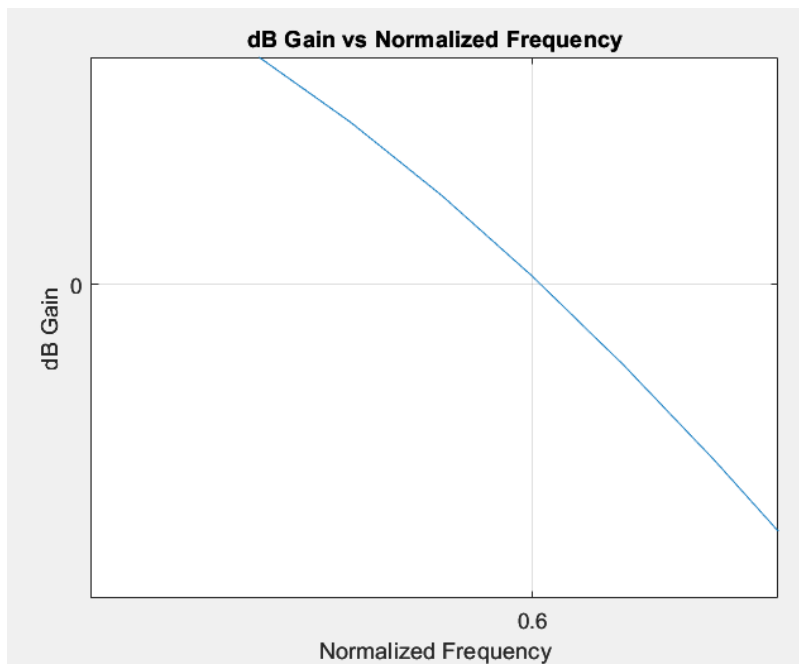


Figure 16 – Criteria for 0.6pi

Next, looking at the 0.65pi to 0.75pi, the curve is between -3 dB and -5 dB as seen in figure 17.

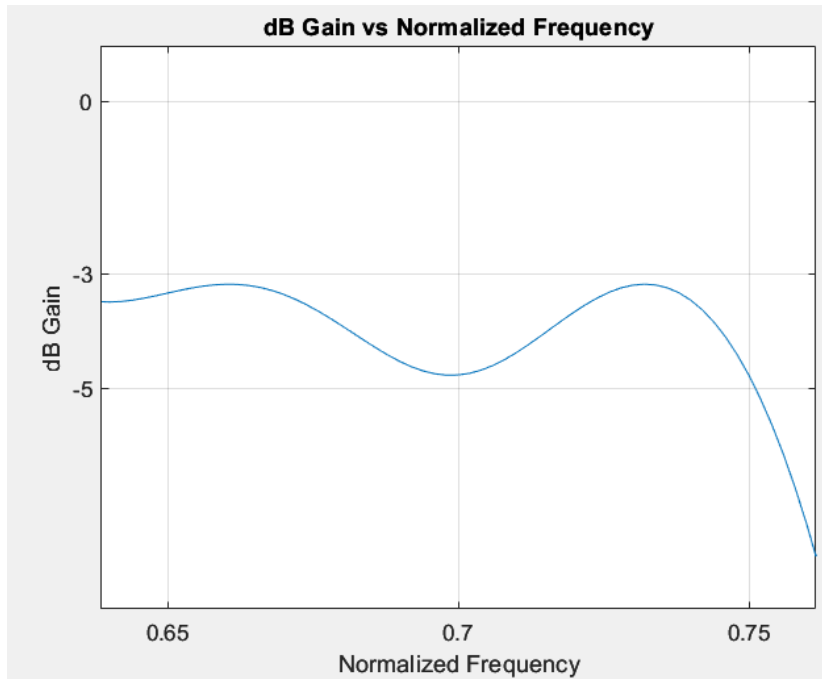


Figure 17 – Criteria between 0.65pi and 0.75pi

Finally, looking at the 0.8pi location, it is evident from figure 18 that the dB gain is below -50 dB.

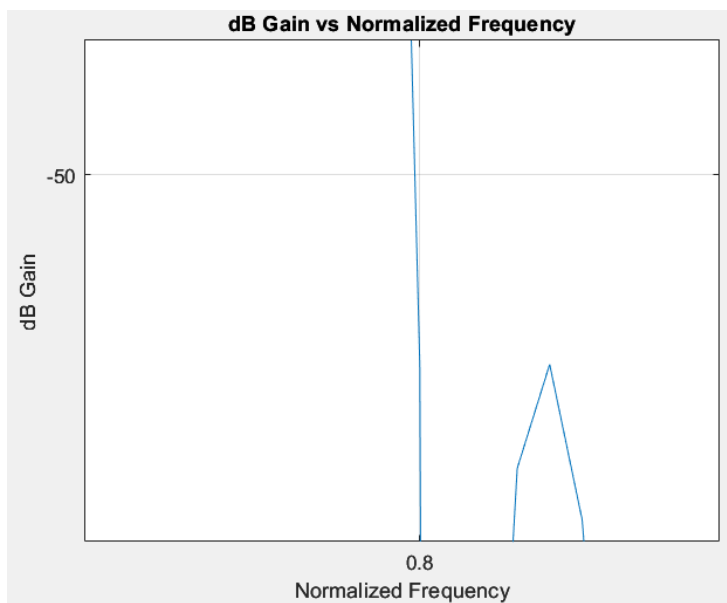


Figure 18 – Criteria for 0.8pi

Thus, this report has shown that with 61 samples the Parks-McClellan filter algorithm is able to meet the requirements of the question. Reducing the samples below 61 samples resulting in the dB conditions not being satisfied in one or more regions, thus 61 samples was

the minimum length needed. Comparing the Hamming filter with the Parks-McClellan, the Parks-McClellan is superior due to less samples (61 vs 93) needed to meet specifications. Furthermore, analyzing curvature, we see that the Parks-McClellan has much more ripples than the Hamming filter, but that this ripples are roughly the same in peak-to-peak amplitude for a given section, hence why the requirements for dB could be met with fewer samples.

Question 1c: IIR Filter

The requirements for the previous two filters will now be implemented with an IIR filter. Specifically, the filter will be a parallel combination of elliptical IIR filters. The code for the IIR elliptical filter is shown below in figure 19.

```

Editor - C:\Users\camer\Documents\MATLAB\Project1c.m
dtfft.m x project1.m x Project1c.m x +
1 [B1,A1] = ellip(5,0.5,65,[0.375 0.625],'bandpass');
2 [B2,A2] = ellip(5,0.5,65,[0.24 0.37],'bandpass');
3 [B3,A3] = ellip(5,0.5,65,[0.63 0.76],'bandpass');
4 C1 = conv(A3,B1);
5 C11 = (10^(0.04))*conv(C1,A2); %first term in num
6 C2 = conv(A3,A1);
7 C22 = (10^(-0.155))*conv(C2,B2); % second term in num
8 C3 = conv(B3,A1);
9 C33 = (10^(-0.155))*conv(C3,A2); %third term in num
10 D1 = conv(A3,A1);
11 D11 = conv(D1, A2); % term in den
12 Num = C11 + C22 + C33; % total num
13 fvtool(Num, D11);

```

Figure 19 - Code for IIR Filter

The dB Gain is plotted in figure 20 with respect to normalized frequency.

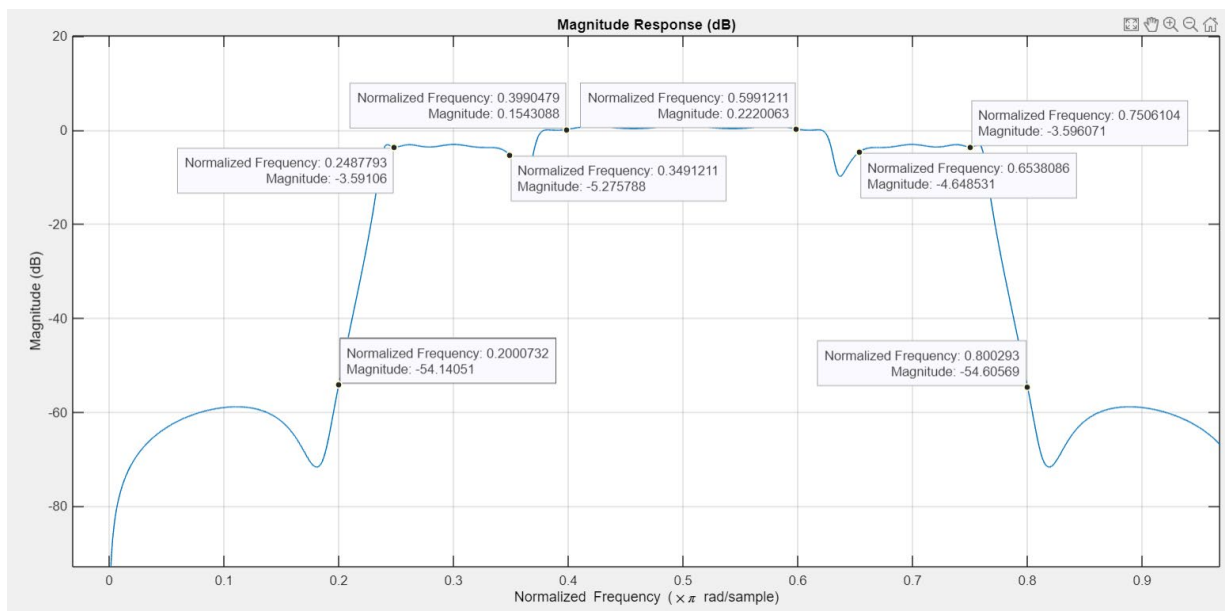


Figure 20 – dB Gain Plot for IIR Filter

From figure 20, it is clear that the requirements for the filter have been met in all of the ranges. Next, the phase response is shown in figure 21.

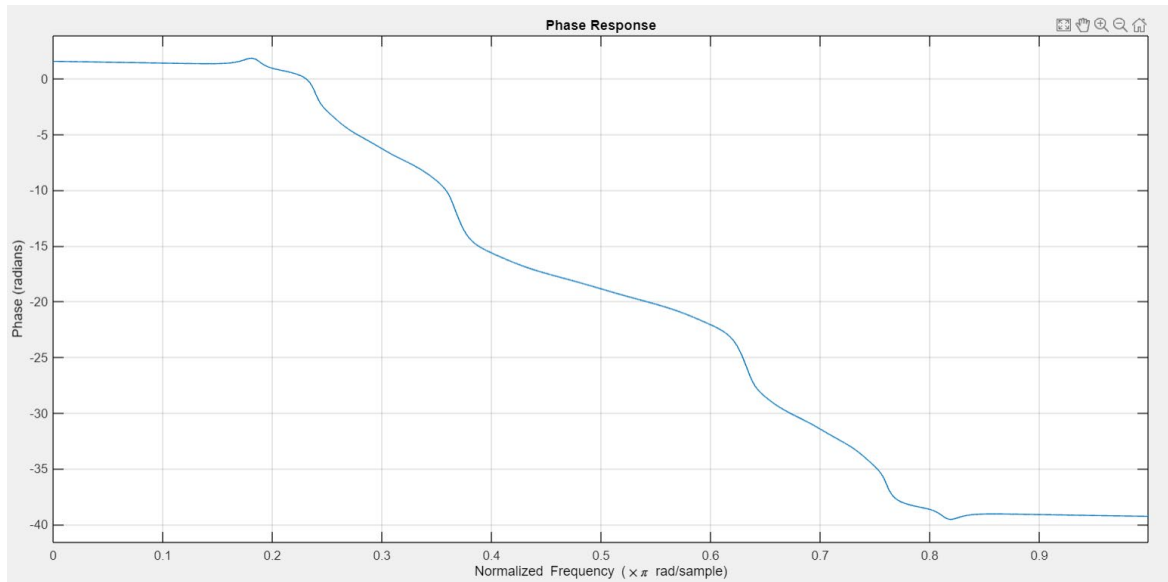


Figure 21 – Phase Response for IIR Filter.

From the phase response plot, we see that this phase is not linear, as was the FIR filters. However, there are some linear regions. For the stop bands between 0 and 0.2π and between 0.8π and π we see that the phase is roughly constant. Furthermore, in all of the passbands, the phase is roughly linear, though with different slopes in each. In the transition bands is where we see the most non-linear behavior as the phase must jump between the stopbands and pass bands. These results are further illustrated in the group delay plot of figure 22.

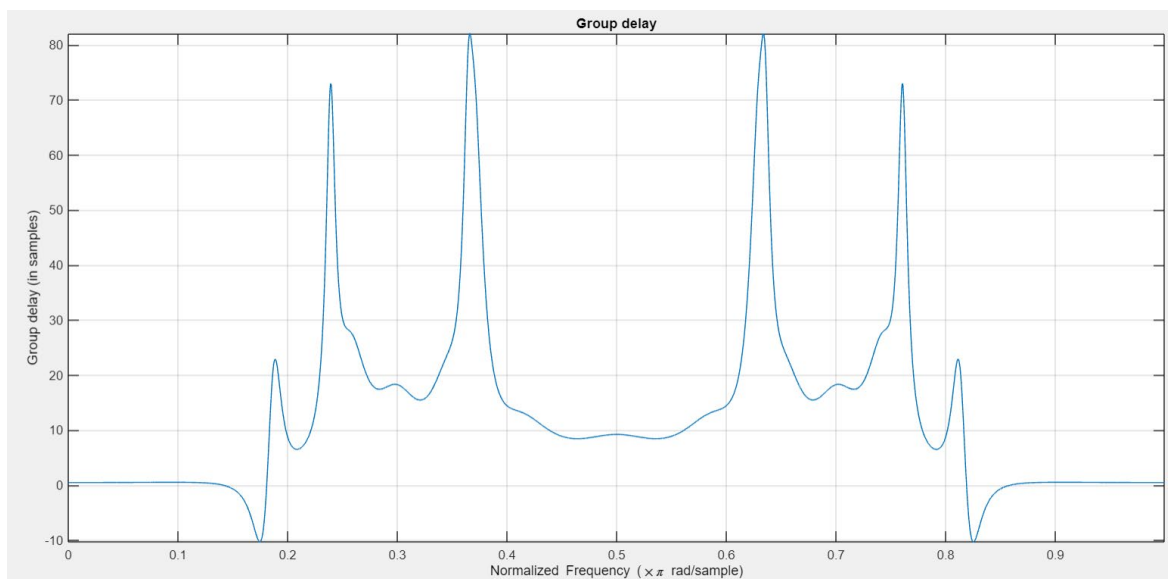
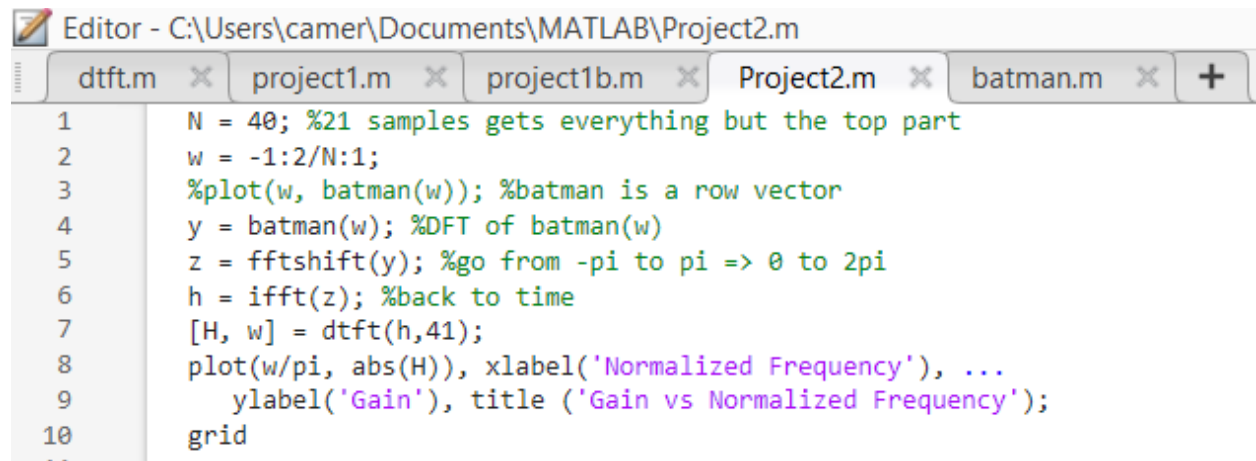


Figure 22 – Group Delay for IIR Filter

From the plot, we see that all of the major spikes in group delay occur in the transition bands. Furthermore, the stopbands are very near linear, whereas the passbands are less linear in nature but still oscillate in a small region of values. Thus, while the IIR filters do not have linear phase, they do approximate phase in all regions except for the transition bands. Furthermore, the IIR filter was able to satisfy the requirements with an order of 5, much lower than the lengths required from the FIR filters.

Question 2: Frequency Sampling w/ IFFT

Question 2 requires students to sample a random DTFT in the shape of a batman, and then utilize the inverse FFT to create an impulse response. Students then utilize the DTFT function in MATLAB to create a DTFT of the impulse response that was previously calculated. This DTFT should match the original DTFT. Utilizing 41 samples, I was able to create a similar curve. Figure 23 shows the code for this process.

The image shows a MATLAB Editor window titled "Editor - C:\Users\camer\Documents\MATLAB\Project2.m". The window has several tabs open: "dtft.m", "project1.m", "project1b.m", "Project2.m", and "batman.m". The "Project2.m" tab is active, displaying the following MATLAB code:

```
1 N = 40; %21 samples gets everything but the top part
2 w = -1:2/N:1;
3 %plot(w, batman(w)); %batman is a row vector
4 y = batman(w); %DFT of batman(w)
5 z = fftshift(y); %go from -pi to pi => 0 to 2pi
6 h = ifft(z); %back to time
7 [H, w] = dtft(h,41);
8 plot(w/pi, abs(H)), xlabel('Normalized Frequency'), ...
9     ylabel('Gain'), title ('Gain vs Normalized Frequency');
10 grid
```

Figure 23 – Code for Question 2

The following batman curve is generated from my code.

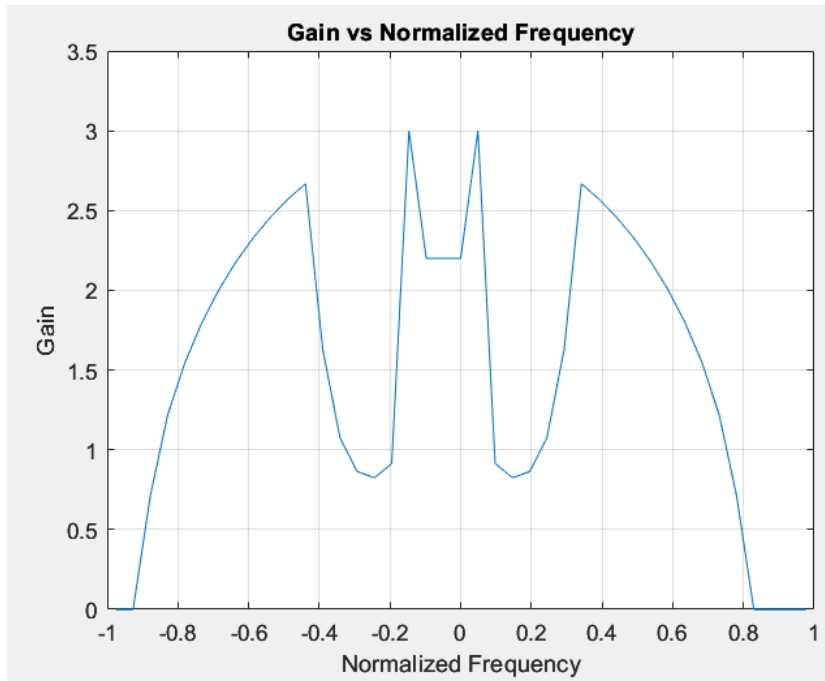


Figure 24 – Batman Graph Generated from Sampling

As we see from figure 24, this curve is a very good representation of the original Batman curve. The signal's peak values are around 2.7 and 3, and the troughs are a little less than 1 just as with the original. For comparison, the original curve is displayed in figure 25.

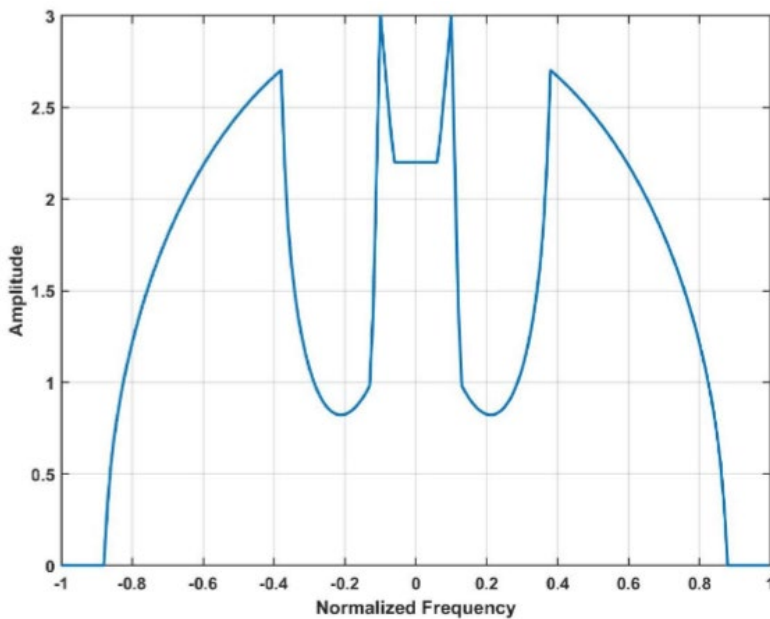


Figure 25 – Original Batman Curve

Furthermore, if the number of samples is reduced to 21, meaning the $N = 20$ and the length of the DTFT is 21, the shape almost resembles a batman, with some slight differences as can be seen in figure 26.

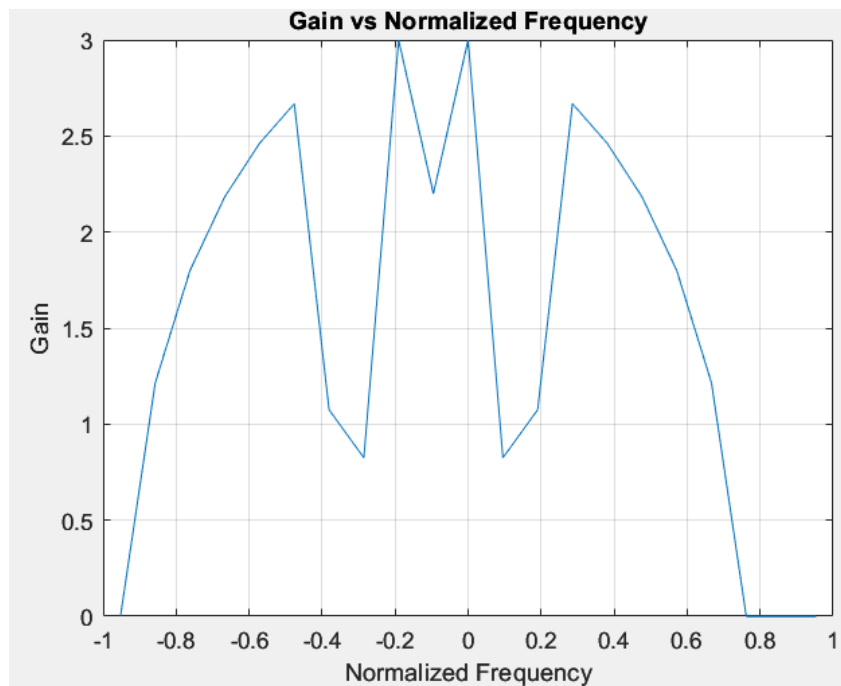


Figure 26 – Batman Curve with 21 Samples

With 21 samples, the amplitudes of the peaks and troughs are still roughly accurate; however, the troughs are not as smooth as before, and the straight line that connected the two peaks in the center of the graph is now gone. Furthermore, the filter goes to zero gain prior to 0.8π , slightly faster than before. Decreasing the length any further caused extreme distortion to the signal, as the amplitudes of the various peaks and troughs began to change, and the filter got less smooth. Thus, with 41 samples, a near perfect representation is achieved, whereas with 21 samples, a decent approximation is achieved.

Analyzing the code, the frequencies are sampled every $2/N$ in the normalized frequency domain as is standard with a DFT. The data is shifted with an FFT shift prior to the inverse FFT, because the IFFT assumes the data is between 0 and 2π . When I did not have the IFFT, the batman curve had a flipped shape. Then, the resulting impulse function goes through a DTFT of the same length. I chose to have the DTFT the same length as the DFT from prior, because if the DTFT was longer, there would be zero padding which would mess up the curve of the graph. Finally, the results are plotted with normalized frequency.

Question 3: Beamformer Design

The first part of this question asks students to design a beamformer using the Parks-McClellan algorithm that passes signals between -20 and -5 degrees with a dB gain between -2 and 2 dB. Furthermore, there are no requirements on the length of the transition band, only that the sidelobes should be as low as possible. Furthermore, the number of antennas is 30, and thus the M is 29 in this problem. With the object of low sidelobes, the transition bands will be long. As the transition bands widen the sidelobes will lessen. However, the purpose is to build a filter focused between -20 and -5 degrees and not say -30 to 5 degrees. Thus, I will present a filter that has a moderate transition band with sidelobes that are still quite low. Hence, I am presenting a filter that maintains both objectives without compromising one or the other. The code for this part of question is given in figure 27.

```

Editor - C:\Users\camer\Documents\MATLAB\Project3a.m
+1 project1.m x project1b.m x Project2.m x batm
1 hpm = firpm(29,[0 0.145 0.25 1],[1 1 0 0],[1 50]);
2 %ideal cut off is 0.131
3 n = -14.5:1:14.5; %30 antennas
4 h = hpm.*exp(-1j*pi*n*0.216); %shift left
5 [H,w] = dtft(h,1000); %dtft
6 deg = (180/pi)*asin(w/pi); %convert to degrees
7 plot(deg,20*log10(abs(H)), xlabel('Degrees'), ...
8      ylabel('Gain'), title ('Gain vs Degrees'));
9 xticks(-80:5:80)
10 grid
11 yticks([-50 -40 -30 -2 0 2])

```

Figure 27 – Code for Parks-McClellan Beamformer Filter

The resulting filter generated is shown in figure 28.

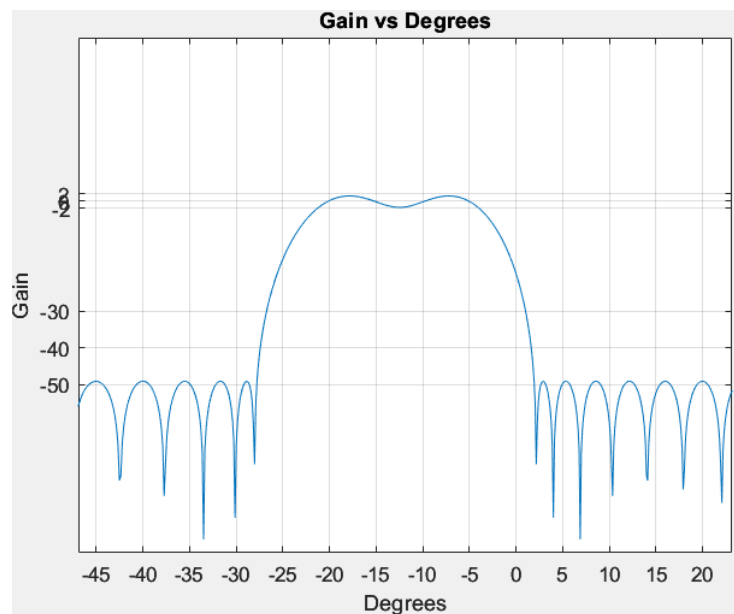


Figure 28 - Parks-McClellan Beamformer Filter

Moreover, figure 29 illustrates the -20 to -5 degree region satisfying the requirements.

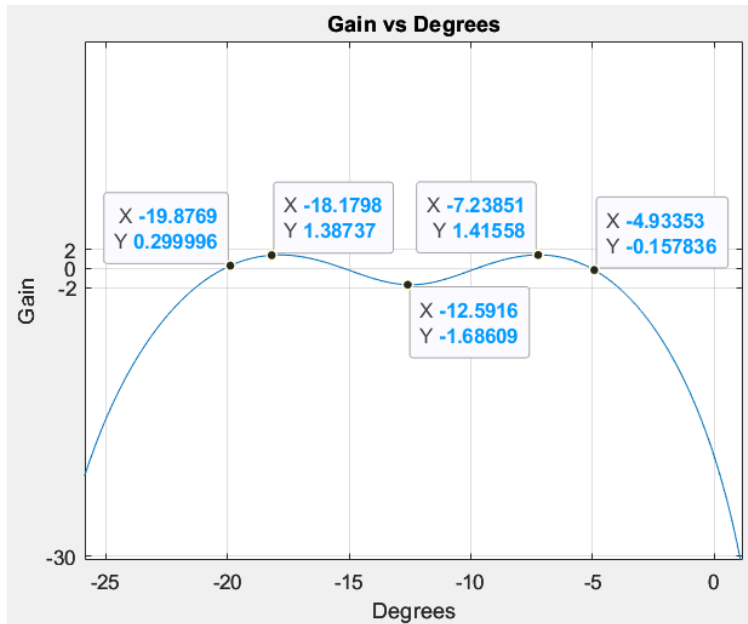


Figure 29 – Passband Region of Parks-McClellan Filter

The second part of this question requires students to shift the filter to the region of 5 to 20 degrees, without utilizing a frequency shift. Besides the frequency shift transformation to the filter, the other quick way to repurpose the previous filter is to perform a time reversal. The discrete time Fourier transform states that if we perform a time reversal, then this will yield a frequency reversal in the normalized frequency domain. The following code is then added to figure 27 to yield figure 30.

Figure 30 – Code for Frequency Reversal of Beamformer Filter

```

Editor - C:\Users\camer\Documents\MATLAB\Project3a.m
dtft.m  project1.m  project1b.m  Project2.m  batman.m  Project3a.m  +
1      hpm = firpm(29,[0 0.145 0.25 1],[1 1 0 0],[1 50]);
2      %ideal cut off is 0.131
3      n = -14.5:1:14.5; %30 antennas
4      h = hpm.*exp(-1j*pi*n*0.216); %shift left
5      hflip = flipplr(h); % time reversal
6      [H,w] = dtft(hflip,1000); %dtft
7      deg = (180/pi)*asin(w/pi); %convert to degrees
8      plot(deg,20*log10(abs(H))), xlabel('Degrees'), ...
9           ylabel('Gain'), title('Gain vs Degrees');
10     xticks(-80:5:80)
11     grid
12     yticks([-50 -40 -30 -2 0 2])

```

As seen above, the only change to the code was the time reversal code on line 5. The “fliplr” function flipped the impulse response values about the y-axis and resulted in the following graph in figure 31.

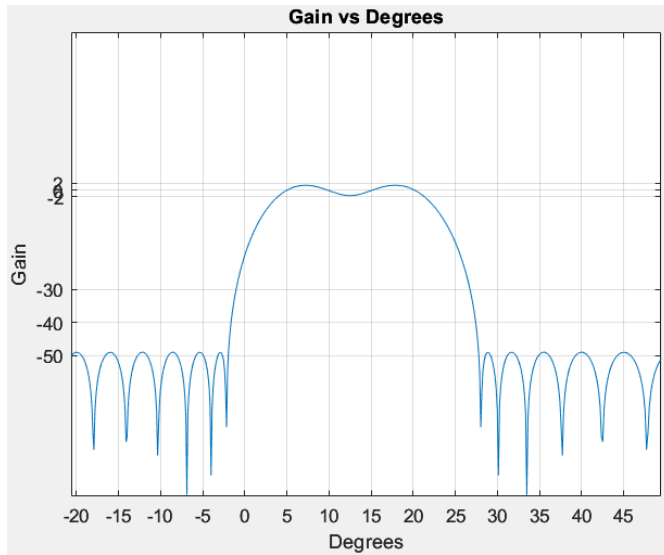


Figure 31 – Plot for Frequency Reversal of Beamformer Filter

The plot above is exactly the same as previously, except for the range of the passband is now between 5 and 20 degrees. Figure 32 illustrates that the passband region satisfies the conditions of the question.

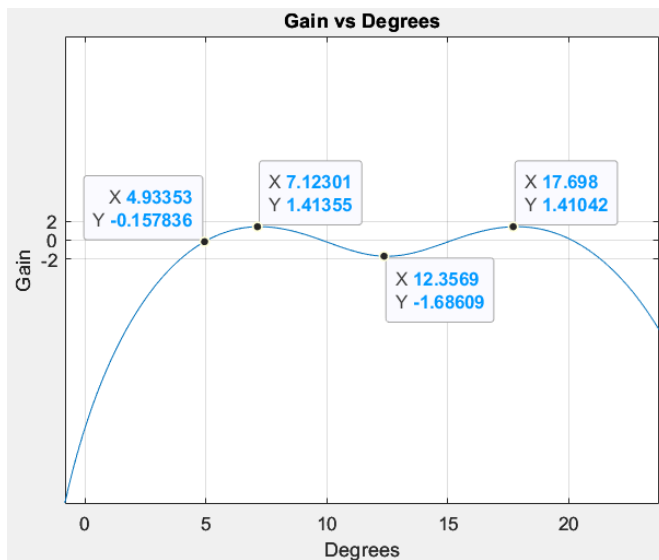


Figure 32 – Plot for Frequency Reversal of Beamformer Filter in Passband Region

The time reversal property is easier to implement than the frequency shift property and displays how very little modification to the code is necessary to change the filter.

The final part of question 3 involves two narrow beams at -20 degrees and -5 degrees. I chose the Hamming window because this window most effectively separated the two beams from each other. The other windows had much less of a magnitude drop between the two

beams and were thus not the preferred choice due to the inability to resolve the two peaks if the beams were moved closer together. The code for these two narrow beams is shown in figure 33.

```

Editor - C:\Users\camer\Documents\MATLAB\project3c.m
dtft.m x project1.m x project1b.m x Project2.m x bat
1      n = -14.5:1:14.5; %30 antennas
2      w1 = -0.342*pi;
3      w2 = -0.087*pi;
4      win = hamming(30); %hamming window
5      h = exp(1j*w1*n)+exp(1j*w2*n);
6      hf = (win').*h; %window impulse
7      [H,w] = dtft(hf,500); %dtft
8      deg = (180/pi)*asin(w/pi); %convert to degrees
9      plot(deg,20*log10(abs(H)), xlabel('Degrees'), ...
10         ylabel('Gain'), title ('Gain vs Degrees'));
11      xticks(-80:5:80)
12      grid

```

Figure 33 – Code for Two Narrow Beams

The results of figure 33 are plotted and displayed in figure 34.

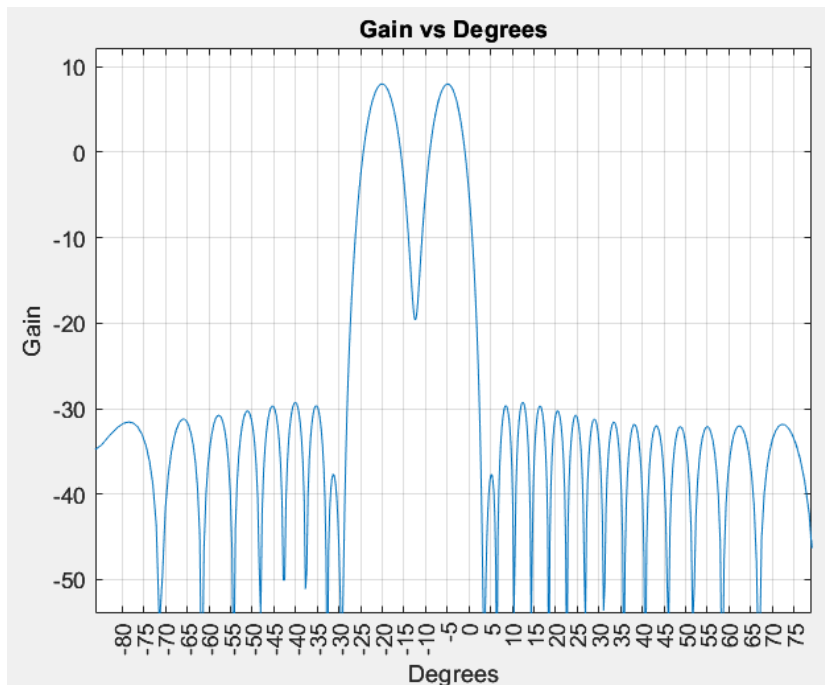


Figure 34 – Graph for Two Narrow Beams

The two beams peak at -20 and -5 degrees as wanted in the directions. Furthermore, the sidelobes are at least 30 dB lower than the beams, which is a satisfactory attenuation.

The second part of the beamformer design prompts students to move the beams closer until the two cannot be resolved. I am using the definition of resolved to be when I can see at least two small peaks. I determined that the two peaks could not be resolved when the exponentials in the time domain have frequencies of -0.275π and -0.155π . These frequencies correspond to the beams having ideal positions of -15.962 and -8.917 degrees respectively. The code involved in this part is shown in figure 35.

```

Editor - C:\Users\camer\Documents\MATLAB\project3cii.m
+3 Project2.m x batman.m x Project3a.m x project
1  n = -14.5:1:14.5; %30 antennas
2  win = hamming(30); %hamming window
3  h = exp(1j*(-0.275*pi)*n)+exp(1j*(-0.155*pi)*n);
4  %-15.962 degrees and -8.917 degrees
5  hf = (win').*h; %window impulse
6  [H,w] = dtft(hf,500); %dtft
7  deg = (180/pi)*asin(w/pi); %convert to degrees
8  plot(deg,20*log10(abs(H))), xlabel('Degrees'), ...
9      ylabel('Gain'), title ('Gain vs Degrees');
10  xticks(-80:5:80)
11  grid

```

Figure 35 – Unresolved Beams in Spatial Filter

The graph of these unresolved beams is shown in figure 36.

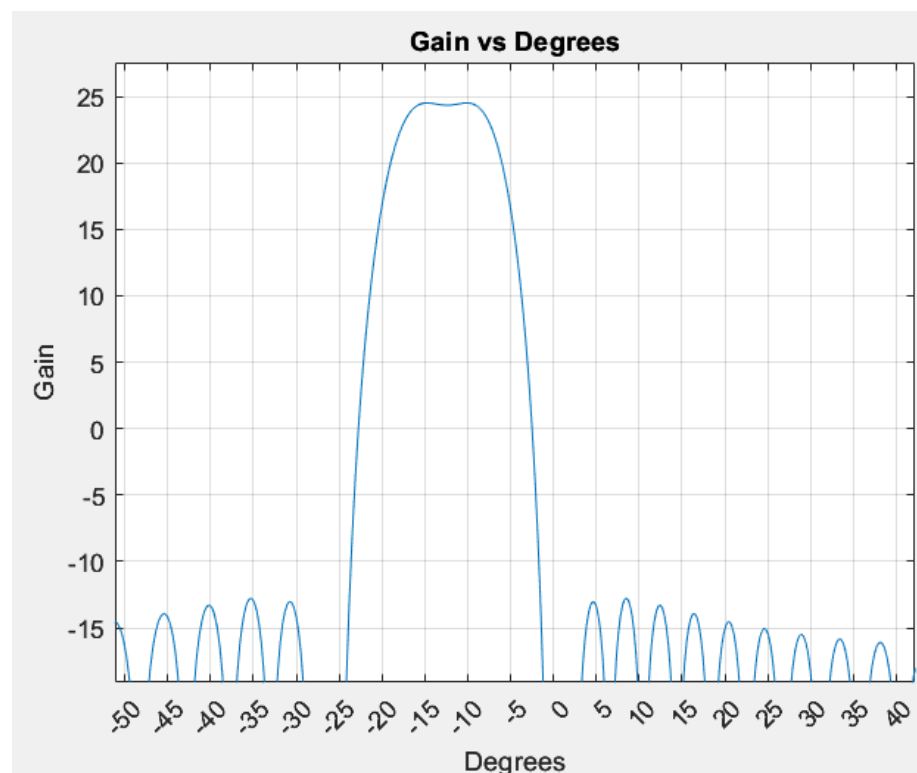


Figure 36 – Plot of Unresolved Beams in Spatial Filter

The last section of the spatial filtering involves creating two beams that are one degree apart. I found that with 251 antennas the two beams can be resolved, but lower than that and the two beams cannot be clearly resolved. I chose the two beams to be at 1 and 2 degrees. The code for this filter is shown in figure 37.

```

Editor - C:\Users\camer\Documents\MATLAB\project3cii.m
+4  batman.m  Project3a.m  project3c.m  project3cii.m
1      n = -125:1:125; %101 antennas
2      win = hamming(251); %hamming window
3      h = exp(1j*(0.017452*pi)*n)+exp(1j*(0.0348995*pi)*n);
4      %1 degree and 2 degrees
5      hf = (win').*h; %window impulse
6      [H,w] = dtft(hf,1350); %dtft
7      deg = (180/pi)*asin(w/pi); %convert to degrees
8      plot(deg,20*log10(abs(H))), xlabel('Degrees'), ...
9          ylabel('Gain'), title('Gain vs Degrees');
10     xticks(-80:5:80)
11     grid

```

Figure 37 – Code for Closely Located Beams

The plot of these closely located beams is displayed in figure 38.

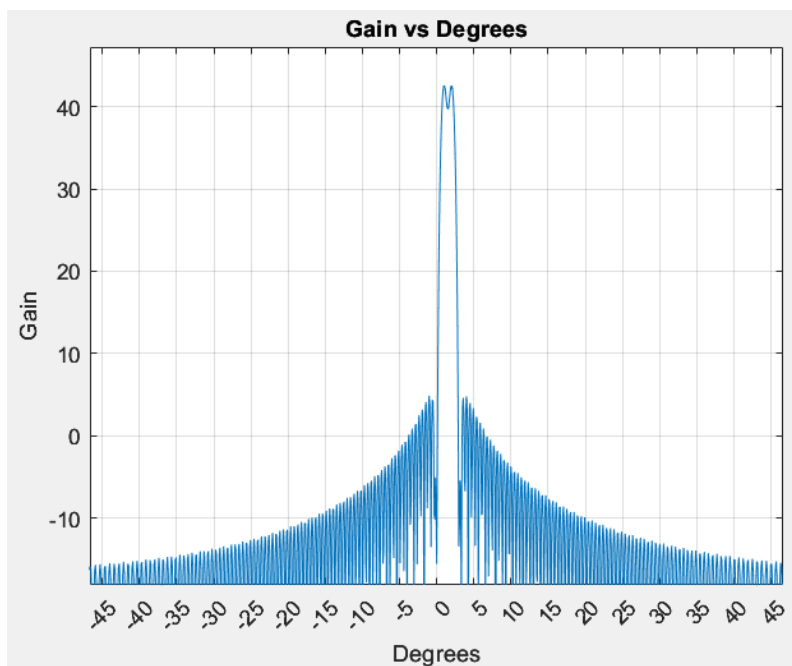


Figure 38 – Code for Closely Located Beams

The two beams can be distinguished in figure 38, thus satisfying the requirements.